

PROGETTO DIDATTICO FULL STACK DEVELOPER

WebApp Hamburgeria
Digital Menu System

DESCRIZIONE GENERALE

- **Descrizione generale del progetto**

- Il progetto consiste nello sviluppo di una **web application per una hamburgeria** che permetta di:
 - Digitalizzare il menù
 - Visualizzare categorie, prodotti e dettagli
 - Mostrare offerte e promozioni
 - Presentare una gallery fotografica
 - Consentire il contatto con il locale

- Il sistema evolve **per fasi**, passando da:

- **Frontend statico con JSON**
- **Backend dinamico con database**
- **API REST**
- **Autenticazione JWT**
- **Deploy containerizzato con Docker**
- I dati di riferimento sono simili a quelli presenti su:
 <https://bourmet.finedelivery.it/menu>

MODULO 1 – DATABASE RELAZIONALI

- **Obiettivi didattici**

- Modellare dati reali
- Normalizzare un database
- Preparare dati per API REST

- **Attività**

- Realizzare un database che permetta di:
- Gestire il catalogo prodotti
- Gestire ingredienti e allergeni
- Gestire preferenze alimentari
- Gestire varianti (150gr / 250gr)
- Generare JSON per API
- Fare controlli di coerenza alimentare

- Creazione **ER diagram**

- Script SQL:

- CREATE TABLE
- INSERT dati demo

- Relazioni:

- 1:N (categorie → prodotti)
- N:M (prodotti ↔ ingredienti)

- **Output**

-  Database MySQL
-  Dati reali importati

DIAGRAMMA ER

- **Entità Principali**
 - **Products**
 - **Categories**
 - **Ingredients**
 - **Allergens**
 - **Dietary Preferences**
 - **Product Variants**
- Product → Categories (N:M)
 - Product → Ingredients (N:M)
 - Ingredient → Allergens (N:M)
 - Product → Dietary Preferences (N:M)
 - Product → Variants (1:N)

PERCHÉ SEPARARE ALLERGENI E DIETARY PREFERENCE?

- Allergeni
 - Obbligo di legge
 - Derivano da ingredienti
 - Sicurezza sanitaria
-
- Dietary Preference
 - Scelta commerciale
 - Attribuiti manualmente
 - Stile alimentare

VARIANTI E ATTRIBUTI

- Servono per evitare di duplicare prodotti
- (non fondamentale)
- Invece di:
 - Burger 150gr
 - Burger 250gr
- Usiamo:
 - products
 - product_variants
- Vantaggi:
 - Struttura scalabile

DATI DA INSERIRE (MINIMO)

- **Categorie**

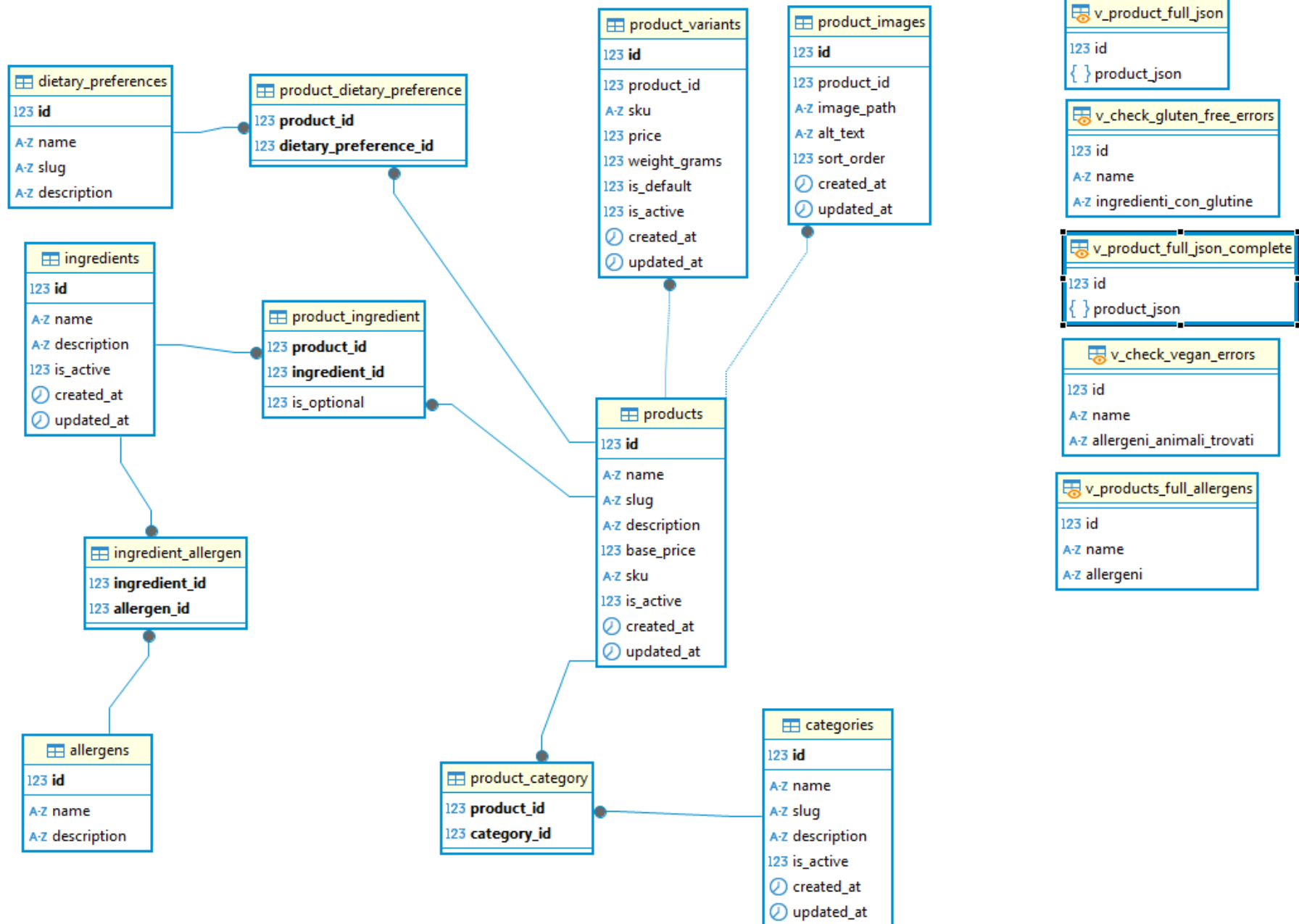
- Servono per organizzare il menu.
- Burger Classici
- Special Burger

- **Allergeni**

- Servono per la sicurezza alimentare.
- Glutine
- Latte
- Uova
- Soia

- **Varianti**

- Servono quando cambia il prezzo.
- Nel nostro caso:
- Peso (150g, 250g, 350g)
- Prezzo associato
-  Se cambia il prezzo → è una variante
-  Se non cambia il prezzo → NON è una variante



VISTE DI CONTROLLO QUALITÀ (VIEW)

- **Controllo Gluten Free**

- Verifica prodotti marcati GF ma contenenti glutine.

- **Controllo Vegano**

- Verifica prodotti vegani ma con latte/uova.

- **Vista Allergeni Aggregati**

- Mostra allergeni per prodotto.

- **Vista API JSON**

- {
 - "nome": "",
 - "descrizione": "",
 - "prezzo_base": "",
 - "categorie": [],
 - "allergeni": [],
 - "ingredienti": [],
 - "varianti": []
- }

ESEMPIO

- 🍔 **Classico Italiano**
- **Hamburger di manzo con cheddar, insalata e pomodoro fresco**
-

- 📋 **Ingredienti**
- Pane brioche
- Hamburger di manzo
- Cheddar
- Insalata
- Pomodoro
-

- ⚠️ **Allergeni**
- Glutine
- Uova
- Latte
-

- 🏷️ **Caratteristiche**
- 🍖 Carne 100% manzo
- ⭐ Best Seller
- us Stile americano

- 🌱 **Vegano Mediterraneo**
- **Burger vegetale con avocado e verdure grigliate**
-

- 📋 **Ingredienti**
- Burger vegetale (soia)
- Insalata
- Pomodoro
- Avocado
-

- ⚠️ **Allergeni**
- Soia
-

- 🌿 **Dietary Preference**
- Vegano
- Vegetariano

MODULO 2 – FIGMA UI/UX

▪ Obiettivi didattici

- UX thinking
- Design system
- Prototipazione

▪ Output

- ✓ File Figma condiviso
- ✓ Design coerente per frontend

▪ Attività

- Analisi utenti:
 - cliente affamato 😊
 - mobile-first
- Wireframe:
 - home
 - lista
 - dettaglio
- UI Design:
 - colori
 - font
 - componenti (card prodotto, badge offerta)
- Prototipo navigabile

MODULO 3 – LINGUAGGI DI MARKUP

▪ Obiettivi didattici

- Strutturare pagine semantiche
- Layout responsive
- Separazione contenuto / stile

▪ Output

-  Frontend statico completo
-  Design responsive

▪ Attività

- Creazione struttura HTML5:
 - header / nav / main / footer

▪ Pagine:

- home.html
- list.html
- detail.html
- offers.html
- gallery.html
- philosophy.html
- contacts.html

▪ CSS:

- mobile first
- grid / flexbox
- componenti riutilizzabili

- Placeholder dinamici (in previsione JS)

MODULO 3 – LINGUAGGI DI MARKUP (FASE 2)

▪ Obiettivi didattici

- Gestione dati JSON
- Rendering dinamico
- Preparazione al backend

▪ Output

-  WebApp funzionante **senza backend**
-  Logica frontend pronta per API

▪ Attività

- Creazione file JSON statici:
 - categories.json
 - products.json
- Fetch dei dati
- Rendering:
 - lista prodotti
 - dettaglio prodotto
- Filtri per categoria

MODULO 5 – BACKEND NODE.JS + JWT + DOCKER

▪ Obiettivi didattici

- API REST
- Sicurezza
- Architettura moderna

▪ Output

- ✓ API REST complete
- ✓ Autenticazione funzionante
- ✓ Ambiente deploy-ready

▪ Attività

- Backend Node.js (Express)
- Connessione DB
- API:
 - GET /categories
 - GET /products
 - GET /products/:id
- Autenticazione JWT:
 - login
 - rotte protette
- Docker:
 - node
 - database
 - docker-compose

PROGRAMMAZIONE MOBILE (.NET MAUI)

- Realizzazione interfacce per lettura dati da api al fine di presentare:
 - lista prodotti
 - dettaglio prodotto
- Filtri per categoria

CONCLUSIONE

- Obiettivo finale del progetto
-  Realizzare una web application completa e professionale, che:
 - Funziona come vero menù digitale
 - È responsive e UX-oriented
 - Usa API REST
 - È containerizzata
 - È presentabile come **project work finale**
- **Competenze acquisite**
 - Lavoro in team
 - Versioning (Git)
 - Comunicazione frontend ↔ backend
 - Approccio enterprise
 - Portfolio professionale
 -
-  Extra (facoltativi)
 - QR Code per accesso menù
 - PWA